

BLOC 3 — CDA CESI — INFCDAAL3

CESIZen

Plan de déploiement, maintenance et sécurisation
Application de santé mentale — Ministère fictif

AUTEUR

Louis Beaujoin

FORMATION

CDA CESI Bloc 3

VERSION

v1.0.0 — Juillet 2026

TESTS CI/CD

48 tests  GitHub Actions

Laravel 13 · PHP 8.3 · Docker · GitHub Actions · Pest 4 · OWASP

Table des matières

1.	Contexte du projet
2.	Plan de déploiement
2.1	Environnements DEV / TEST / PROD
2.2	Stack technique
2.3	Infrastructure Docker (environnement TEST configuré)
2.4	Pipeline CI/CD et pilotage
3.	Plan de sécurisation
3.1	Matrice des vulnérabilités et calculs de criticité
3.2	Actions correctives et préventives
3.3	Solutions de chiffrement et cryptage
3.4	Gestion de crise — escalade et communication
3.5	Données personnelles et conformité RGPD
3.6	Bonnes pratiques de développement
4.	Plan de maintenance et pérennité
4.1	Outil de gestion — GitHub Issues & Projects
4.2	Méthodologie et SLA
4.3	Veille technologique — Dependabot
5.	Tests automatisés — 48 tests · 122 assertions
6.	Procédure de rollback
7.	Conclusion

CESiZen est une plateforme web de bien-être mental développée pour un Ministère fictif dans le cadre du Bloc 3 (CDA CESI — INFCDAAL3). Elle propose des exercices de respiration guidés, des pages d'information sur la santé mentale, un espace personnel pour les utilisateurs connectés et un back-office d'administration complet.

Objectif du Bloc 3

Concevoir un plan de déploiement complet, mettre en place un environnement de déploiement avec automatisations, assurer la maintenance via un outil de ticketing, et sécuriser l'application en identifiant les vulnérabilités avec calculs de criticité.

Modules implémentés

Module	Statut	Fonctionnalités principales
Comptes utilisateurs	✓ Obligatoire	Inscription, connexion, profil, changement MDP, désactivation (admin)
Informations	✓ Obligatoire	Pages santé mentale, CRUD admin, accès public
Exercices de respiration	✓ Au choix	3 exercices prédéfinis + exercice personnalisable, historique sessions
API REST	Bonus	Authentification par token Bearer, accès aux exercices

2.1 Environnements DEV / TEST / PROD

Critère	DEV (WAMP local)	TEST (Docker)	PROD (théorique)
Objectif	Développement quotidien	Validation intégration client	Mise en service réelle
Démarrage	<code>composer dev</code>	<code>docker compose up --build</code>	Pipeline CD automatisé
Port	8000	8080	443 (HTTPS — Let's Encrypt)
Base de données	MySQL WAMP local	MySQL 8 container isolé	MySQL managé (RDS ou serveur dédié)
Assets Vite	Hot reload actif	Compilés dans l'image	Compilés + CDN
APP_DEBUG	true	false	false
Ressources min.	4 Go RAM	4 Go RAM · 3 Go disque	2 vCPU · 4 Go RAM · 50 Go SSD
Environnement configuré	—	✓ Déployé et opérationnel	Décrit théoriquement

2.2 Stack technique

Backend

- Laravel **13** (LTS)
- PHP **8.3**
- MySQL **8.0**
- Apache 2.4 (Docker)

Frontend

- Tailwind CSS **4**
- Vite **7**
- Blade templates
- AlpineJS (animations respiration)

Tests & Qualité

- Pest **4** — 48 tests · 122 assertions
- SQLite in-memory (CI)
- GitHub Actions CI/CD

Infrastructure

- Docker Compose
- Git + GitHub
- Dependabot
- SemVer v1.0.0

2.3 Infrastructure Docker — Environnement TEST configuré

L'environnement TEST est entièrement conteneurisé. C'est l'environnement réellement mis en place et configuré pour les démonstrations.

Service	Image	Port exposé	Rôle
app	Build local (PHP 8.3 Apache + Node 20)	8080 → 80	Application Laravel + Apache
db	mysql:8.0	Non exposé (réseau interne)	Base de données persistante + healthcheck

```
#!/bin/bash
set -e
# Crée .env depuis .env.example si absent (secrets hors image – bonne pratique)
[ ! -f .env ] && cp .env.example .env
# Génère la clé applicative si absente
[ -z "$APP_KEY" ] && php artisan key:generate --force
# Attend MySQL (healthcheck) puis migre automatiquement
until php artisan migrate --force 2>&1; do
    echo "Base de données non disponible, nouvelle tentative dans 3s..."; sleep 3
done
# Données de démo si DB_SEED=true
if [ "${DB_SEED:-false}" = "true" ]; then php artisan db:seed --force; fi
exec apache2-foreground
```

Accès de démonstration — <http://localhost:8080>

Admin : admin@cesizen.fr / password | Utilisateur : user@cesizen.fr / password

2.4 Pipeline CI/CD et pilotage

Le pipeline GitHub Actions assure l'automatisation complète du cycle de vie du code. Il est déclenché à chaque push sur `main` ou `develop`.

Job	Déclencheur	Étapes	Critère go/no-go
tests	Push main / develop	PHP 8.3 · Composer · SQLite · <code>php artisan test</code>	48/48 tests passent
docker	<code>needs: tests</code>	<code>docker build -t cesizen-app .</code>	Build sans erreur

Critère go/no-go

Si un seul test échoue dans le job `tests`, le job `docker` est annulé automatiquement. Aucun code cassé ne peut atteindre l'environnement de test.

Pilotage et supervision de l'environnement déployé

Le pilotage de l'application déployée repose sur plusieurs niveaux :

Niveau	Outil	Indicateurs surveillés
Logs applicatifs	<code>docker compose logs app</code> · <code>storage/logs/laravel.log</code>	Erreurs PHP, exceptions, requêtes échouées
Disponibilité	Monitoring HTTP (uptime check)	Code HTTP 200, temps de réponse
CI/CD	GitHub Actions — onglet Actions	Statut des runs, durée, taux de réussite
Dépendances	Dependabot (GitHub Security)	CVE détectées, mises à jour disponibles
Versioning	Git log · Tags SemVer	Historique déploiements, rollback possible

En production, un outil de monitoring dédié (UptimeRobot, Datadog ou Sentry) serait configuré pour alerter en temps réel en cas de dégradation de service.

Versioning des sources — Git Flow + SemVer

```
main      ← code stable, livré en production – tag SemVer v1.0.0
├── develop ← intégration continue des développements
│   ├── feature/* ← fonctionnalité isolée, créée depuis develop
│   └── ...
└── ...

Flux : feature/* → PR → develop → PR → main → git tag v1.0.0
```

Convention de commit : **Conventional Commits** (`feat:` `fix:` `docs:` `test:` `chore:`). Chaque merge sur `main` est taggé selon **Semantic Versioning** : MAJEUR.MINEUR.PATCH.

3.1 Matrice des vulnérabilités et calculs de criticité

Méthode : **Criticité** = **Probabilité (P)** × **Impact (I)**, échelle 1–5 pour chaque dimension.

Échelle de probabilité (P)

- P1 : Très improbable (<10%)
- P2 : Improbable (10–30%)
- P3 : Possible (30–60%)
- P4 : Probable (60–80%)
- P5 : Très probable (>80%)

Interprétation du score

- 1–4 : **FAIBLE**
- 5–9 : **MOYEN**
- 10–15 : **ÉLEVÉ**
- 16–25 : **CRITIQUE**

#	Vulnérabilité	Réf. OWASP	P	I	Criticité	Niveau	Traitement
V01	Brute-force sur /connexion	A07:2021	4	4	16	CRITIQUE	throttle:5,1 → HTTP 429 ✓
V02	Exposition des secrets (.env)	A05:2021	3	5	15	ÉLEVÉ	.gitignore + .dockerignore ✓
V03	Mode debug en production	A05:2021	3	4	12	ÉLEVÉ	APP_DEBUG=false ✓
V04	Injection XSS (paramètres JS)	A03:2021	3	4	12	ÉLEVÉ	cast (int) + Blade {{ }} + CSP ✓
V05	Mots de passe faibles / compromis	A02:2021	3	4	12	ÉLEVÉ	bcrypt 12 rounds ✓
V06	Injection SQL	A03:2021	2	5	10	ÉLEVÉ	Eloquent ORM (requêtes préparées) ✓
V07	Accès non autorisé routes admin	A01:2021	2	5	10	ÉLEVÉ	Middleware auth + rôles ✓
V08	Contrefaçon de requête CSRF	A01:2021	3	3	9	MOYEN	Token @csrf sur tous formulaires ✓
V09	Vol de session utilisateur	A07:2021	2	4	8	MOYEN	Sessions sécurisées Laravel ✓
V10	Clickjacking (iframe)	A05:2021	2	3	6	MOYEN	X-Frame-Options: SAMEORIGIN ✓

V11	MIME sniffing	A05:2021	2	2	4	FAIBLE	X-Content-Type-Options: nosniff ✓
V12	Chargement ressources tierces	A05:2021	2	3	6	MOYEN	Content-Security-Policy: default-src 'self' ✓

Couverture : 12/12 vulnérabilités traitées — 100% (objectif grille : 75%)

Toutes les vulnérabilités identifiées ont une action corrective implémentée et testée (48 tests automatisés dont 2 tests de sécurité dédiés au SecurityHeadersMiddleware).

3.2 Actions correctives et préventives

Vulnérabilité	Action corrective (déjà implémentée)	Action préventive
V01 — Brute-force	throttle:5,1 sur /connexion et /api/login	Monitoring des logs 429 ; alertes automatiques
V02 — Secrets exposés	.env dans .gitignore + .dockerignore ; .env.example sans valeurs	Audit régulier du git log ; GitHub Secret Scanning actif
V03 — Mode debug	APP_DEBUG=false dans .env.example	Vérification avant chaque déploiement prod
V04 — XSS	cast (int) dans BreathingController ; Blade {{ }} auto-escape ; CSP	Code review obligatoire sur toute sortie HTML
V05 — MDP faibles	bcrypt 12 rounds via cast 'hashed' sur modèle User	Politique de complexité à ajouter en production
V06 — SQL injection	Eloquent ORM avec requêtes préparées PDO	Pas de concaténation SQL manuelle — code review
V07 — Accès admin	Middleware auth sur toutes les routes protégées ; rôles admin/user	Tests d'accès dans la suite Pest (AuthTest)
V08 — CSRF	@csrf token sur tous les formulaires POST	Vérification automatique par le middleware Laravel
V09 — Vol session	Sessions Laravel chiffrées (APP_KEY) ; expiration automatique	Session httpOnly et secure en HTTPS prod
V10 — Clickjacking	X-Frame-Options: SAMEORIGIN	CSP frame-ancestors 'none' en complément
V11 — MIME sniffing	X-Content-Type-Options: nosniff	Content-Type explicite sur toutes les réponses API
V12 — Ressources tierces	Content-Security-Policy: default-src 'self'	Durcissement progressif (suppression unsafe-inline)

3.3 Solutions de chiffrement et cryptage

Donnée / Canal	Solution	Détail technique
Mots de passe	Hachage bcrypt	12 rounds, salage automatique, irréversible. Cast <code>'hashed'</code> sur le modèle User.
Sessions et cookies	Chiffrement AES-256-CBC	Laravel chiffre automatiquement les données de session avec l'APP_KEY (256 bits).

Données en transit (PROD)	TLS 1.3 — HTTPS	Certificat Let's Encrypt (gratuit, renouvellement automatique). En TEST : HTTP local (réseau fermé).
Clé applicative	APP_KEY — 32 octets aléatoires	Générée par <code>php artisan key:generate</code> , stockée dans .env (hors git).
Tokens API	Hash SHA-256	Tokens stockés hashés en base (Sanctum / champ api_token).

HTTPS en production

En environnement TEST Docker (local), les échanges sont en HTTP car le réseau est fermé et non exposé à Internet. En production réelle, HTTPS avec TLS 1.3 et HSTS est obligatoire.

3.4 Gestion de crise — Escalade, responsabilités et communication

Ce plan définit la réponse à tout incident de sécurité avéré (attaque, violation de données, vulnérabilité critique). Il suit le cycle **Détecter** → **Classer** → **Confiner** → **Corriger** → **Communiquer** → **Post-mortem**.

Processus de réponse à incident

- H+0** **Détection.** Monitoring des logs Apache/PHP/MySQL, alerte GitHub Security (Dependabot), signalement utilisateur via le template "Incident de sécurité" sur GitHub Issues.
- H+15 min** **Classification.** Le responsable technique qualifie l'incident selon la criticité (voir tableau ci-dessous) et ouvre le ticket GitHub Issues avec le label approprié.
- H+30 min** **Notification interne.** Escalade selon la criticité (voir matrice d'escalade). Pour P1 Critique : Direction notifiée, DPO contacté si des données personnelles sont impliquées.
- H+1h** **Confinement.** Mise en maintenance (page 503), révocation des sessions actives (`php artisan auth:clear-resets`), blocage de l'IP attaquante au niveau Apache/firewall.
- H+3h (P1)** **Correction.** Branche `hotfix/CVE-XXXX` depuis `main` , correction, PR → revue → merge, tag SemVer patch (v1.0.1). CI/CD rebuild automatique.
- H+4h (P1)** **Retour à la normale.** Levée de la maintenance, vérification de l'intégrité des données, tests de non-régression.
- <72h** **Communication externe (si violation RGPD).** Notification à la CNIL dans les 72h. Email aux utilisateurs concernés. Message de transparence sur le site.
- J+7** **Post-mortem.** Rapport documenté dans le ticket GitHub. Identification des causes racines. Actions préventives supplémentaires. Mise à jour du plan de sécurisation.

Matrice d'escalade

Criticité	Définition	Responsable principal	Notifications	Délai correction
P1 Critique	Service indisponible ou violation de données	Responsable technique	Direction + DPO si RGPD	< 3h ouvrées
P2 Majeur	Fonctionnalité principale compromise	Développeur	Responsable technique	< 24h ouvrées

Communication externe — Messages types

Pendant l'incident (site en maintenance)

"CESIZen est temporairement indisponible pour maintenance urgente de sécurité. Nous travaillons à rétablir le service dans les plus brefs délais. Nous vous informerons dès que le service est rétabli."

Après résolution (email utilisateurs si RGPD)

"Cher utilisateur, nous avons détecté et corrigé un incident de sécurité le [DATE]. Vos données [description]. Nous avons [actions prises]. Si vous avez des questions : [contact]. La CNIL a été notifiée conformément à l'article 33 du RGPD."

Obligation RGPD — Article 33

En cas de violation de données personnelles : notification à la CNIL sous **72 heures**. En cas d'impact sur les droits et libertés des personnes : notification aux utilisateurs concernés sous **72 heures** (Article 34).

3.5 Données personnelles et conformité RGPD

Principe RGPD	Mise en œuvre dans CESIZen	Outil / Mécanisme
Minimisation des données (Art. 5)	Seuls nom, email, MDP hashé collectés. Aucun tracking, aucune donnée de santé réelle.	Modèle User — champs fillable limités
Conservation limitée (Art. 5)	Comptes inactifs >3 ans supprimés. Sessions expirées >90 jours purgées.	Commande artisan planifiée (PROD)
Droit d'accès (Art. 15)	Profil complet consultable dans l'espace personnel	Route /profil (utilisateur connecté)
Droit de rectification (Art. 16)	Modification nom, email, mot de passe depuis le profil	Formulaire de mise à jour du profil
Droit à l'effacement (Art. 17)	Désactivation + suppression de compte par l'administrateur — supprime toutes les données associées	Back-office admin → action de suppression
Sécurité des données (Art. 32)	Bcrypt 12 rounds, HTTPS prod, APP_DEBUG=false, secrets hors git	SecurityHeadersMiddleware + .env.example
Notification violation (Art. 33/34)	Procédure définie : CNIL <72h, utilisateurs <72h	Plan de gestion de crise (section 3.4)

Limitation pédagogique

Ce projet étant éducatif, il n'y a pas de DPO désigné ni de politique de confidentialité publiée. En production réelle, ces éléments seraient obligatoires (Art. 37 RGPD — DPO, Art. 13 — politique de confidentialité).

3.6 Bonnes pratiques de développement

Pratique	Outil / Convention	Valeur ajoutée
Versioning sémantique	SemVer (MAJEUR.MINEUR.PATCH)	Lisibilité immédiate de l'ampleur d'un changement
Messages de commit	Conventional Commits	CHANGELOG auto-générable, historique lisible
Branches	Git Flow simplifié (main/develop/feature/*)	Isolation des développements, PR traçables
Code review	Pull Requests GitHub avec template checklist	Validation tests + sécurité avant merge
Documentation	README.md complet, dossier technique PDF	Onboarding rapide, traçabilité des décisions

Secrets	.gitignore + .dockerignore + .env.example	Aucun secret dans l'historique git ou l'image Docker
Dépendances	Dependabot hebdomadaire	Veille CVE automatisée, PRs de mise à jour
Tests automatisés	Pest 4 — 48 tests en CI	Régressions détectées avant intégration

4.1 Outil de gestion — GitHub Issues & Projects

L'outil de ticketing retenu est **GitHub Issues** couplé à un board **GitHub Projects**. Ce choix centralise tout dans le même outil que le code source et le CI/CD, garantissant la traçabilité complète (ticket → commit → PR → merge).

Composant	Configuration	Acteurs
GitHub Issues	Labels : bug, amélioration, bloquant-critique, bloquant-majeur, mineur, sécurité, RGPD	Prestataire (dev) + Client (Ministère)
GitHub Projects	Board Kanban 4 colonnes : Backlog → En cours → En revue → Terminé	Prestataire (pilotage)
Templates Issues	<code>bug_report.md</code> + <code>feature_request.md</code> pré-remplis	Client (saisie guidée)
PR Template	Checklist obligatoire : tests · sécurité · revue · SemVer	Prestataire (validation)
Dependabot	PRs automatiques Composer + npm hebdo, GitHub Actions mensuel	Automatique (veille)

Cycle de vie d'un ticket — Prestataire ↔ Client

1. **Signalement** : Le client ouvre un ticket via le template GitHub Issues (étapes de reproduction, criticité, environnement)
2. **Qualification** : Le prestataire labellise le ticket (<1h pour bloquant critique) et l'ajoute au board
3. **Traitement** : Branche feature/ ou fix/ créée, développement, tests passants
4. **Revue** : Pull Request ouverte, checklist validée, code review
5. **Livraison** : Merge main, tag SemVer, fermeture automatique du ticket (`Fixes #XX`)
6. **Validation client** : Démonstration sur l'environnement TEST Docker

4.2 Méthodologie et SLA

Criticité	Définition	Prise en compte	Correction	Exemple concret
Bloquant critique	Service indisponible / perte données	< 1h	< 3h ouvrées	Erreur 500 sur /connexion
Bloquant majeur	Fonctionnalité principale dégradée	< 4h	< 24h ouvrées	Exercice de respiration planté

4.3 Veille technologique — Dependabot

Dependabot surveille automatiquement les dépendances et crée des PRs GitHub à chaque mise à jour disponible. Processus de validation :

1. Dependabot crée une PR avec la mise à jour
2. GitHub Actions déclenche les 48 tests sur la PR
3. Le développeur vérifie le CHANGELOG de la dépendance
4. Si les tests passent et le CHANGELOG ne signale pas de rupture → merge
5. Si rupture : migration adaptée avant merge

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "composer"
    directory: "/"
    schedule: { interval: "weekly" }
  - package-ecosystem: "npm"
    directory: "/"
    schedule: { interval: "weekly" }
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule: { interval: "monthly" }
```

Résultat final — 7 juillet 2026

Tests : **48 passés** (0 échoué) · Assertions : **122** · Durée : ~2.5s · CI GitHub Actions :  success

Suite	Fichier	Tests	Couverture
Unit / Modèles	BreathingExerciseModelTest	3	getCycleDuration, scope active, relation
Unit / Modèles	UserModelTest	4	isAdmin, hash password, fillable
Unit / Modèles	InformationPageModelTest	2	scope published, scope ordered
Feature / Auth	AuthTest	13	Inscription, connexion, profil, admin, permissions
Feature / Respiration	BreathingTest	12	CRUD exercices, sessions, paramètres custom
Feature / Information	InformationTest	6	CRUD pages, accès public/admin
Feature / Sécurité	SecurityTest	4	En-têtes HTTP (+ CSP), throttle 429, XSS
Exemples	ExampleTest × 2	4	Sanity checks
TOTAL		48	122 assertions · 2.5s · 0 échec

Niveau 1 — Rollback du code

```
# Option A : Annuler un commit (préserve l'historique)
git revert <hash-du-commit-problématique>
git push origin main

# Option B : Retour à un tag stable
git checkout v1.0.0 && git push origin main --force
```

Niveau 2 — Rollback base de données

```
php artisan migrate:rollback
mysql -u root -p cesizen_prod < backup_avant_migration.sql
```

Scénario	Délai déclenchement	Action
Erreur 500 en production	Immédiat (<5 min)	Rollback code + redémarrage
Régression fonctionnelle majeure	<30 min après constat	Rollback code
Corruption de données	Immédiat	Rollback BDD depuis backup
Vulnérabilité critique (CVE)	Immédiat + maintenance	Rollback + patch hotfix

Déploiement

- 3 environnements décrits (DEV/TEST/PROD)
- Docker Compose opérationnel (port 8080)
- Pipeline CI/CD GitHub Actions (2 jobs)
- Rollback documenté

Sécurité

- 12 vulnérabilités analysées (matrice P×I)
- 12/12 actions correctives implémentées
- 5 en-têtes HTTP de sécurité (+ CSP)
- Plan de gestion de crise formalisé

Maintenance

- GitHub Issues + Projects configurés
- SLA formalisé par criticité
- Dependabot veille hebdomadaire
- Cycle prestataire ↔ client documenté

Qualité

- 48 tests automatisés (122 assertions)
- Conventional Commits + SemVer
- Git Flow structuré
- RGPD Art. 5, 15, 16, 17, 32, 33, 34

État final — 7 juillet 2026

- 48/48 tests passent en CI (GitHub Actions )
- Application Docker : <http://localhost:8080> (données démo disponibles)
- Code : github.com/louisbeaujoin/cesizen · PR #17 develop → main ouverte